

Off-Policy Actor-Critic Methods

Daniil Zelezetsky

Higher School of Economics

March 4, 2026

В ПРОШЛЫХ ЛЕКЦИЯХ: Подходы к обучению агента

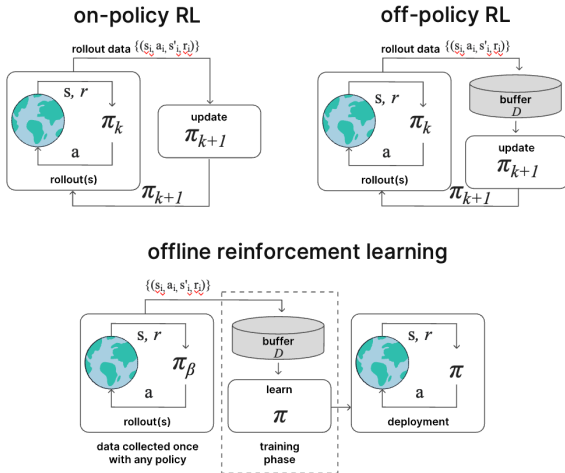
1 Off-Policy RL:

Собираем отложенный опыт, на котором учимся (DQN, DDPG, SAC)

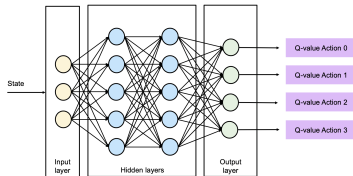
2 On-Policy RL:

Обучаем стратегию по мере исследования среды (REINFORCE, AC, PPO)

3 Offline RL: Учимся на оффлайн датасете экспертных данных (обычный sequence modeling)



В ПРОШЛЫХ ЛЕКЦИЯХ: DQN



Temporal Difference Error:

$$TD(\theta) = \frac{1}{N} \sum_{(s_t, a_t, r_t, s_{t+1}) \in D} \left[\left(r_t + \gamma \max_{a'} Q^{\theta^-}(s_{t+1}, a') - Q^{\theta}(s_t, a_t) \right)^2 \right]$$

В ПРОШЛЫХ ЛЕКЦИЯХ: ON-POLICY AC

1. Генерируем данные взаимодействия со средой (Rollout):

- Внутри эпизода :
 - Наблюдаем s_t
 - Выбираем $a_t \sim \pi_\theta(a_t|s_t)$
 - Получаем r_t, s_{t+1}
 - Оцениваем $Q(s_t, a_t)$ через критика

2. Обновление параметров актора:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot Q(s_t, a_t)$$

3. Считаем loss критика:

$$L_{critic} = (Q_\theta(s_t, a_t) - (r_t + \gamma Q_\theta(s_{t+1}, a_{t+1})))^2$$

4. Обновляем веса критика:

$$\theta^{new} = \theta^{old} - \alpha \nabla_\theta L_{critic}(\theta^{old})$$

Off-Policy Actor–Critic: основные идеи и разделение ролей

Две функции:

- **Critic:** аппроксимирует $Q(s, a)$ или $V(s)$
- **Actor:** параметризует стратегию $\pi_\theta(a|s)$ или $\mu_\theta(s)$

Обновление критика (TD-обучение):

$$L(\phi) = \mathbb{E}[(Q_\phi(s, a) - y)^2]$$

$$y = r + \gamma Q_\phi(s', a')$$

Обновление актора:

$$\nabla_\theta J = \mathbb{E}\left[\nabla_a Q_\phi(s, a)\Big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right]$$

Off-Policy обучение и Replay Buffer

On-policy:

- Обучение на данных текущей стратегии
- Требуется новых траекторий после каждого обновления

Off-policy:

- Обучение на данных, собранных любой стратегией
- Используется **Replay Buffer** $D = \{(s, a, r, s')\}$

Преимущества:

- Повышение sample efficiency
- Стабилизация обучения
- Возможность многократного использования опыта

Общая схема Off-Policy Actor–Critic:

- 1 Собираем переходы в буфер
- 2 Обучаем critic через TD
- 3 Обновляем actor через градиент по Q

DDPG: мотивация и основная идея

Цель: off-policy actor–critic для непрерывных действий.

Проблема Q-learning:

$$y = r + \gamma \max_{a'} Q(s', a') \Rightarrow \max_{a'} \text{сложно в continuous space}$$

Идея DDPG:

- Ввести детерминированного актора $a = \mu_{\theta}(s)$
- Критик аппроксимирует $Q_{\phi}(s, a)$
- Тогда “максимизация” заменяется подстановкой:

$$\max_a Q(s, a) \approx Q(s, \mu_{\theta}(s))$$

Обучение: critic обучаем TD-методом на replay buffer, actor — градиентом через Q .

Deterministic Policy Gradient: обновление актора

Актор: $a = \mu_\theta(s)$

Целевая функция:

$$J(\theta) = \mathbb{E}_{s \sim D} [Q_\phi(s, \mu_\theta(s))]$$

Градиент (chain rule):

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim D} \left[\nabla_a Q_\phi(s, a) \Big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s) \right]$$

Важно:

- градиент идёт *через критика* по действию a
- обучение off-policy: состояния s берём из replay buffer D

Критик DDPG: TD-цель и target networks

Критик: $Q_\phi(s, a)$

TD-цель:

$$y = r + \gamma Q_{\phi^-}(s', \mu_{\theta^-}(s'))$$

Лосс критика:

$$L(\phi) = \mathbb{E}_{(s,a,r,s') \sim D} \left[(Q_\phi(s, a) - y)^2 \right]$$

Soft update target сетей:

$$\phi^- \leftarrow \tau \phi + (1 - \tau) \phi^- \quad \theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

где обычно $\tau \ll 1$.

Исследование: шум для exploration

DDPG использует **детерминированного** актора, поэтому exploration нужно задавать отдельно.

Поведение (behavior policy):

$$a_t = \mu_{\theta}(s_t) + \epsilon_t$$

Где ϵ_t — шум (часто гауссовский), иногда используют процесс Орнштейна–Уленбека.

Что важно на практике:

- шум влияет на качество покрытия состояний в replay buffer
- слишком большой шум $\Rightarrow$ плохие данные и нестабильность
- слишком маленький шум $\Rightarrow$ застревание в локальных режимах

DDPG: псевдокод (высокоуровнево)

Algorithm 1 DDPG (high-level)

- 1: Инициализировать actor μ_θ , critic Q_ϕ
 - 2: Инициализировать target сети $\theta^- \leftarrow \theta$, $\phi^- \leftarrow \phi$
 - 3: Инициализировать replay buffer D
 - 4: **for** каждого шага взаимодействия **do**
 - 5: выбрать действие $a = \mu_\theta(s) + \epsilon$, env.step, сохранить (s, a, r, s') в D
 - 6: семплировать минибатч из D
 - 7: $y \leftarrow r + \gamma Q_{\phi^-}(s', \mu_{\theta^-}(s'))$
 - 8: обновить critic: минимизировать $(Q_\phi(s, a) - y)^2$
 - 9: обновить actor: максимизировать $Q_\phi(s, \mu_\theta(s))$
 - 10: soft update: ϕ^-, θ^-
 - 11: **end for**
-

Типичные симптомы:

- $Q_\phi(s, a)$ начинает систематически завышаться
- actor начинает эксплуатировать ошибки критика $\Rightarrow$ деградация политики
- чувствительность к гиперпараметрам (LR , τ , шум, размер буфера)

TD3: мотивация

DDPG хорошо работает в continuous control, но часто **нестабилен**.

Основная проблема: критик начинает **переоценивать** $Q(s, a)$, а актор затем эксплуатирует эти ошибки:

$$a = \mu_{\theta}(s) \Rightarrow \mu_{\theta} \text{ максимизирует } Q_{\phi}(s, a)$$

TD3 (Fujimoto et al., 2018): три улучшения DDPG:

- 1 **Twin critics** + clipped double Q-learning (борьба с переоценкой Q)
- 2 **Delayed** обновление актора
- 3 **Target policy smoothing** (снижение чувствительности к ошибкам обновления)

1) Twin critics и clipped double Q-learning

В TD3 два критика:

$$Q_{\phi_1}(s, a), \quad Q_{\phi_2}(s, a)$$

Target (clipped):

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i^-}(s', a')$$

Лосс каждого критика:

$$L(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim D} \left[(Q_{\phi_i}(s, a) - y)^2 \right], \quad i \in \{1, 2\}$$

Интуиция:

- в DDPG используется один критик $\Rightarrow$ завышение цели
- минимум из двух оценок даёт **консервативную** оценку

2) Target policy smoothing

Цель в TD-обучении зависит от действия на следующем состоянии. Если критик имеет локальные артефакты по a , актер может некорректно обучиться на них.

Сглаживание target действия:

$$a' = \text{clip}_{[a_{\min}, a_{\max}]} (\mu_{\theta^-}(s') + \epsilon), \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

часто дополнительно:

$$\epsilon \leftarrow \text{clip}_{[-c, c]}(\epsilon)$$

Интуиция:

- мы оцениваем не одно точечное действие, а что-то в окрестности
- уменьшаем влияние ошибок аппроксимации критика по действию

3) Delayed обновление актора

Критики обновляются на каждом шаге градиента, а актор примерно в два раза реже

Почему это помогает:

- актор опирается на градиенты $\nabla_a Q_{\phi_1}(s, a)$
- если критик ещё неточен, обновления актора усиливают ошибки
- отложенные обновления дают критикам время обучиться сильнее

На практике актор обновляют раз в d шагов (часто $d = 2$):

$$\theta \leftarrow \theta + \eta \nabla_{\theta} \mathbb{E}_{s \sim D} [Q_{\phi_1}(s, \mu_{\theta}(s))]$$

TD3: полный update-цикл

1) Target действие (smoothing):

$$a' = \text{clip}(\mu_{\theta^-}(s') + \text{clip}(\epsilon)), \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

2) Target (clipped double Q):

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i^-}(s', a')$$

3) Обновление критиков:

$$\phi_i \leftarrow \arg \min_{\phi_i} \mathbb{E} \left[(Q_{\phi_i}(s, a) - y)^2 \right]$$

4) Delayed обновление актора:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s \sim D} [Q_{\phi_1}(s, \mu_{\theta}(s))]$$

(выполняется раз в d шагов)

5) Soft update target сетей:

$$\phi_i^- \leftarrow \tau \phi_i + (1 - \tau) \phi_i^-, \quad \theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

DDPG:

- один критик, target без сглаживания
- актер обновляется так же часто, как критик

TD3 добавляет:

- 2 критика и $\min(\cdot)$ в TD-цели
- шум в target действии (policy smoothing)
- реже обновляет актора (delayed)

Результат:

- меньше переоценка Q
- выше стабильность и предсказуемость обучения
- обычно сильнее DDPG на тех же задачах continuous control

Algorithm 2 TD3

- 1: Инициализировать актор μ_θ , критики Q_{ϕ_1}, Q_{ϕ_2}
 - 2: Инициализировать target сети $(\theta^-, \phi_1^-, \phi_2^-)$
 - 3: Инициализировать replay buffer D
 - 4: **for** каждого шага взаимодействия **do**
 - 5: собрать переход и сохранить в D
 - 6: семплировать минибатч из D
 - 7: $a' \leftarrow \text{policy smoothing}(\mu_{\theta^-}(s'))$
 - 8: $y \leftarrow r + \gamma \min(Q_{\phi_1^-}(s', a'), Q_{\phi_2^-}(s', a'))$
 - 9: обновить оба критика по MSE y
 - 10: **if** шаг кратен d **then**
 - 11: обновить актора, максимизируя $Q_{\phi_1}(s, \mu_\theta(s))$
 - 12: soft update target сетей
 - 13: **end if**
 - 14: **end for**
-

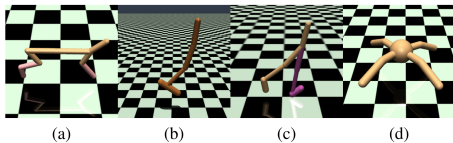
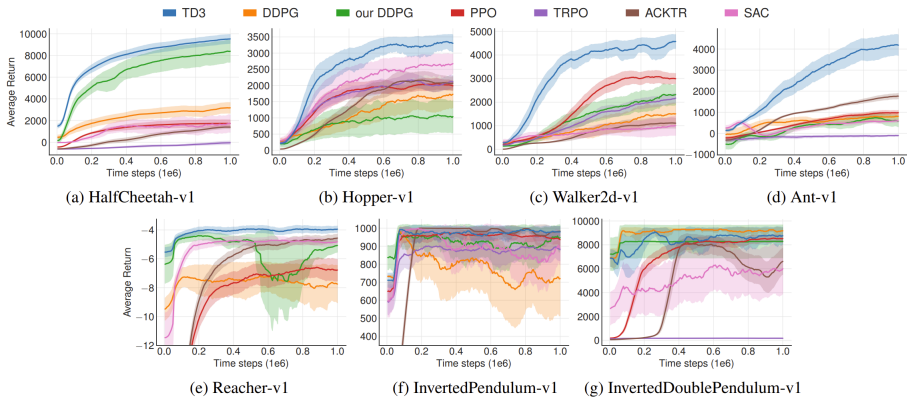


Figure 4. Example MuJoCo environments (a) HalfCheetah-v1, (b) Hopper-v1, (c) Walker2d-v1, (d) Ant-v1.



SAC: от детерминированной к стохастической политике

DDPG / TD3:

$$a = \mu_{\theta}(s)$$

SAC использует **стохастического актора**:

$$a \sim \pi_{\theta}(a|s)$$

SAC объединяет:

- off-policy обучение
- replay buffer D
- twin critics (как в TD3)
- энтропийную регуляризацию

Ключевая идея: максимизируем reward и энтропию политики.

Максимизация энтропии

Стандартная задача RL:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

В SAC:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right]$$

где

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a|s)]$$

α — коэффициент температуры (баланс reward / entropy).

Soft Bellman-уравнение

Определим soft Q-функцию:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s', a' \sim \pi} [Q^\pi(s', a') - \alpha \log \pi(a'|s')]$$

В реализации SAC используем два критика:

$$Q_{\phi_1}(s, a), \quad Q_{\phi_2}(s, a)$$

TD-цель:

$$y = r + \gamma \left(\min_{i=1,2} Q_{\phi_i^-}(s', a') - \alpha \log \pi_\theta(a'|s') \right)$$

где $a' \sim \pi_\theta(\cdot|s')$.

Лосс каждого критика:

$$L(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim D} \left[(Q_{\phi_i}(s, a) - y)^2 \right], \quad i \in \{1, 2\}$$

Здесь:

$$y = r + \gamma \left(\min_{j=1,2} Q_{\phi_j^-}(s', a') - \alpha \log \pi_{\theta}(a'|s') \right)$$

Это расширение TD3 с добавлением энтропийного члена.

Обновление актора

Актор минимизирует:

$$J_{\pi}(\theta) = \mathbb{E}_{s \sim D, a \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a|s) - \min_{i=1,2} Q_{\phi_i}(s, a) \right]$$

Интерпретация:

- максимизируем Q
- одновременно увеличиваем энтропию

Актор балансирует между exploitation и exploration.

Reparameterization trick

Чтобы дифференцировать через стохастическую политику:

$$a \sim \pi_{\theta}(a|s)$$

переписываем как:

$$a = f_{\theta}(s, \epsilon), \quad \epsilon \sim \mathcal{N}(0, I)$$

Часто:

$$a = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \epsilon)$$

Это позволяет получать градиент напрямую по θ .

Автоматическая настройка α

Температура α обучается отдельно:

$$J(\alpha) = \mathbb{E}_{a \sim \pi_\theta} [-\alpha (\log \pi_\theta(a|s) + \mathcal{H}_{\text{target}})]$$

Идея:

- задаём target entropy
- если энтропия слишком мала $\rightarrow \alpha$ увеличивается
- если слишком велика $\rightarrow \alpha$ уменьшается

Это делает алгоритм менее чувствительным к гиперпараметрам.

Algorithm 3 SAC (high-level)

- 1: Инициализировать актора π_θ
 - 2: Инициализировать критиков Q_{ϕ_1}, Q_{ϕ_2}
 - 3: Инициализировать target критиков
 - 4: Инициализировать replay buffer D
 - 5: **for** every step **do**
 - 6: собрать переход и сохранить в D
 - 7: семплировать минибатч
 - 8: вычислить target y
 - 9: обновить оба критика
 - 10: обновить актора
 - 11: обновить α (если используется)
 - 12: soft update target критиков
 - 13: **end for**
-

SAC: полный update-цикл (формулы)

1) Семплирование минибатча:

$$(s, a, r, s') \sim D$$

2) Семплирование следующего действия:

$$a' \sim \pi_{\theta}(\cdot | s')$$

3) Target для критиков:

$$y = r + \gamma \left(\min_{i=1,2} Q_{\phi_i^-}(s', a') - \alpha \log \pi_{\theta}(a' | s') \right)$$

4) Обновление критиков:

$$\phi_i \leftarrow \arg \min_{\phi_i} \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(Q_{\phi_i}(s, a) - y \right)^2 \right], \quad i \in \{1, 2\}$$

SAC: update актора и температуры

5) Обновление актора:

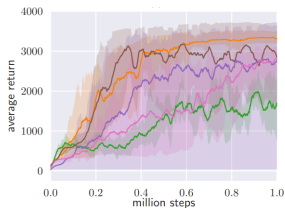
$$\theta \leftarrow \arg \min_{\theta} \mathbb{E}_{s \sim D, a \sim \pi_{\theta}} \left[\alpha \log \pi_{\theta}(a|s) - \min_{i=1,2} Q_{\phi_i}(s, a) \right]$$

6) Обновление температуры (если используется):

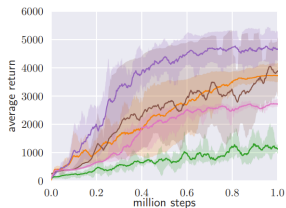
$$\alpha \leftarrow \arg \min_{\alpha} \mathbb{E}_{a \sim \pi_{\theta}} [-\alpha (\log \pi_{\theta}(a|s) + \mathcal{H}_{\text{target}})]$$

7) Soft update target критиков:

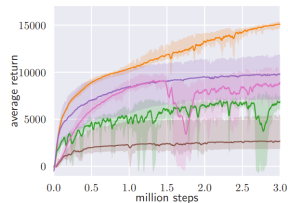
$$\phi_i^- \leftarrow \tau \phi_i + (1 - \tau) \phi_i^-, \quad i \in \{1, 2\}$$



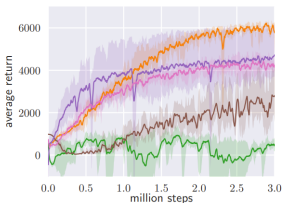
(a) Hopper-v1



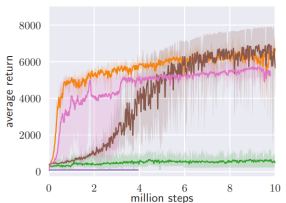
(b) Walker2d-v1



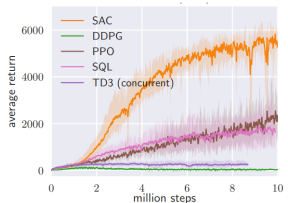
(c) HalfCheetah-v1



(d) Ant-v1



(e) Humanoid-v1



(f) Humanoid (rllab)